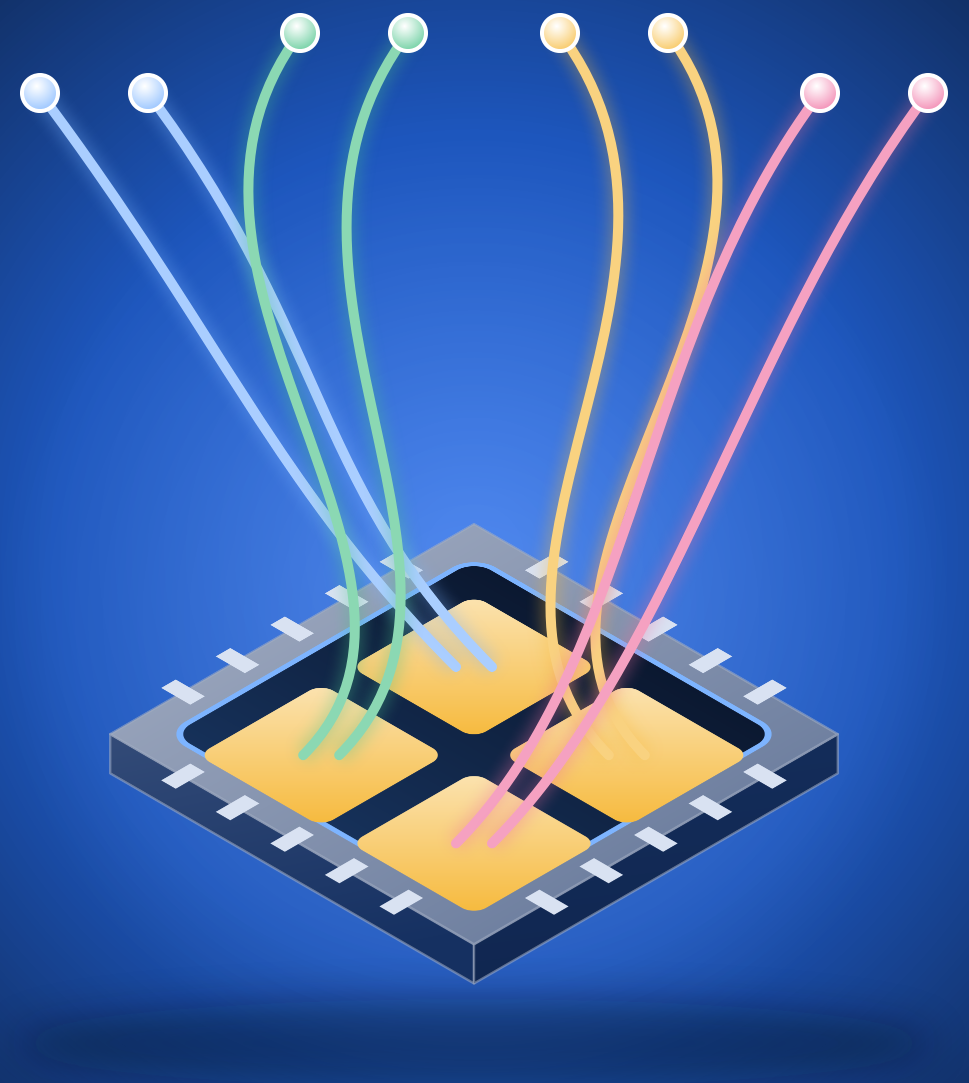
Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · WEEK 4

Threads and Multicore Programming

Several threads of control inside one process, programming for multicore systems, and the libraries that make it practical.

Heng Rathpisey · Department GIC · Academic year 2025–2026



Learning objectives

1. Identify the basic parts of a thread, and contrast threads with processes.
2. Describe the benefits and the challenges of multithreaded programs.
3. Explain implicit threading: thread pools, fork-join and Grand Central Dispatch.
4. Describe how Windows and Linux represent threads.
5. Write multithreaded programs with the Pthreads, Java and Windows APIs.

THIS WEEK

Topic 4

Week 3 assumed one thread per process. Almost no modern program works that way.

DISCUSSION

How does a word processor check spelling while you type?

It reads keys, draws text, checks spelling and saves in the background. One process. How many things at once?

PART 1

Why threads

A cheaper way to do several things at once than creating several processes.

Motivation

Applications are multithreaded

Most modern applications run several tasks inside one process: update the display, fetch data, check spelling, answer a network request.

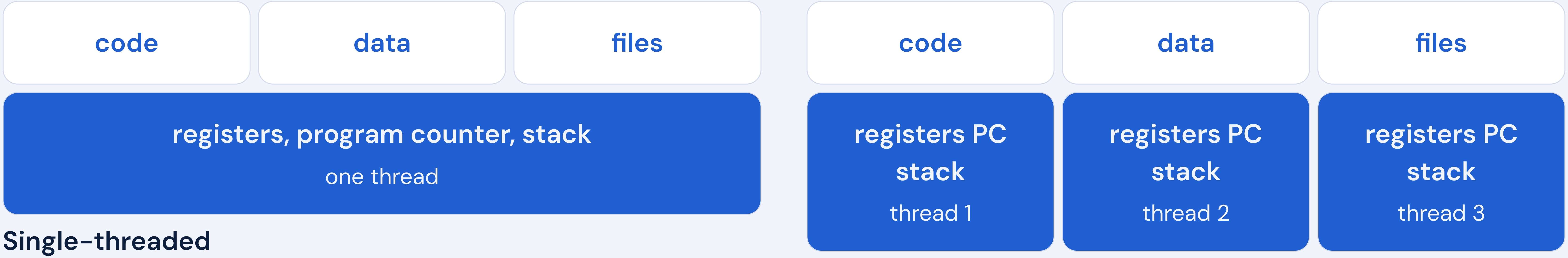
Creation is light

Creating a process is heavy on time and resources. Creating a thread inside an existing process is light.

Kernels are multithreaded

The kernel itself runs many threads, for managing devices, memory and interrupts.

Single-threaded and multithreaded processes



code
data
files
registers, program counter, stack
one thread

Single-threaded

code
data
files
registers PC stack
thread 1
registers PC stack
thread 2
registers PC stack
thread 3

Multithreaded

Threads of one process share its code, data and open files. Each thread has its own thread ID, program counter, registers and stack.

A multithreaded server

Client sends a request →

Server creates a thread for the request →

New thread serves this client →

Server goes back to listening

Without threads the server would serve one client at a time, or create a whole process per request. A thread per request is far cheaper.

Benefits

Responsiveness

The program continues to run even if part of it is blocked. Essential for user interfaces.

Resource sharing

Threads share the memory and resources of their process, with no shared memory or message passing to set up.

Economy

Creating a thread is cheaper than creating a process, and switching threads is cheaper than a context switch.

Scalability

A multithreaded process can use several cores at once.

Process and thread compared

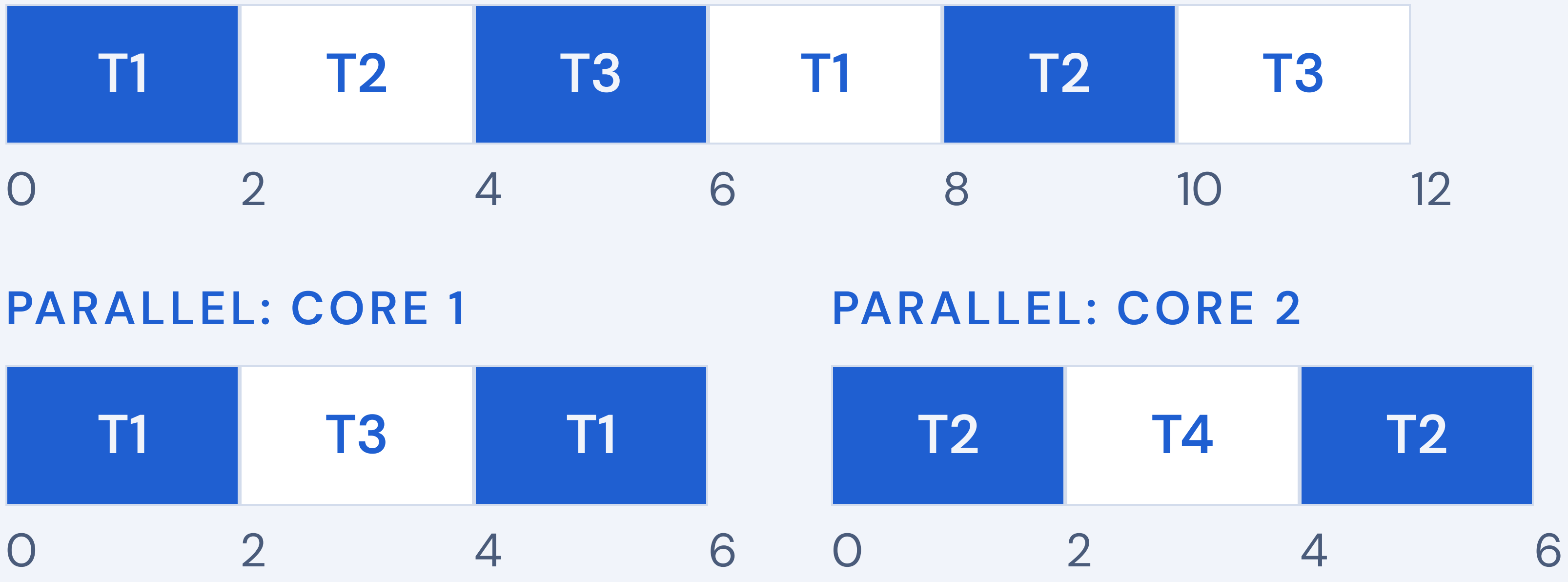
	Process	Thread
Address space	Its own	Shared with the other threads of the process
Creation	Heavy: memory must be set up	Light: only a stack and registers
Switching	A full context switch	Faster, inside the same address space
Communication	Needs IPC	Direct, through shared data
A fault	Affects only that process	Can bring down the whole process

PART 2

Multicore programming

More cores only help if the program is written to use them.

Concurrency and parallelism



CONCURRENT: ONE CORE, TASKS TAKE TURNS

T1
T2
T3
T1
T2
T3
0
2
4
6
8
10
12

PARALLEL: CORE 1

T1
T3
T1
0
2
4
6

PARALLEL: CORE 2

T2
T4
T2
0
2
4
6

Concurrency: more than one task makes progress. Parallelism: more than one task runs at the same instant. A single core gives concurrency without parallelism.

Programming challenges

Dividing activities

Find the parts of the work that can run as separate tasks.

Balance

Make sure the tasks do work of about equal value.

Data splitting

Divide the data so each task can work on its own part.

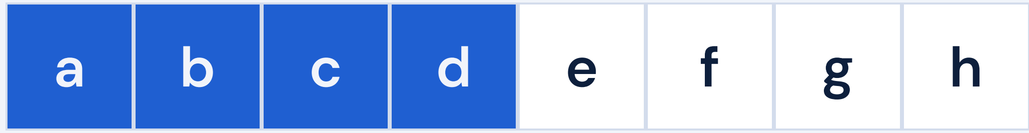
Data dependency

When one task needs another's result, their execution must be synchronized.

Testing and debugging

Many execution paths are possible, and a failure may not repeat.

Data parallelism and task parallelism



Data parallelism

The same operation on different parts of the data. Core 1 sums the first half of an array while core 2 sums the second half.

Core 1

sorts the data

Core 2

finds the maximum

Task parallelism

Different operations, each done by its own thread, on the same data or on different data.

Amdahl's law

$$\text{speedup} \leq 1 / (S + (1 - S) / N)$$

S is the fraction of the program that must run serially. N is the number of cores.

As N grows, the speedup approaches 1 / S. The serial part sets the limit, however many cores we add.

Serial part S	2 cores	4 cores	Limit
10%	1.82	3.08	10
25%	1.60	2.29	4
50%	1.33	1.60	2

With a quarter of the program serial, four cores give 2.29 times the speed, and no number of cores gives more than 4 times.

PART 3

Multithreading models

How the threads a program sees are mapped onto the threads the kernel schedules.

User threads and kernel threads

ABOVE THE KERNEL

User threads

Managed by a thread library in user space, without kernel support. The three main libraries are POSIX Pthreads, Windows threads and Java threads.

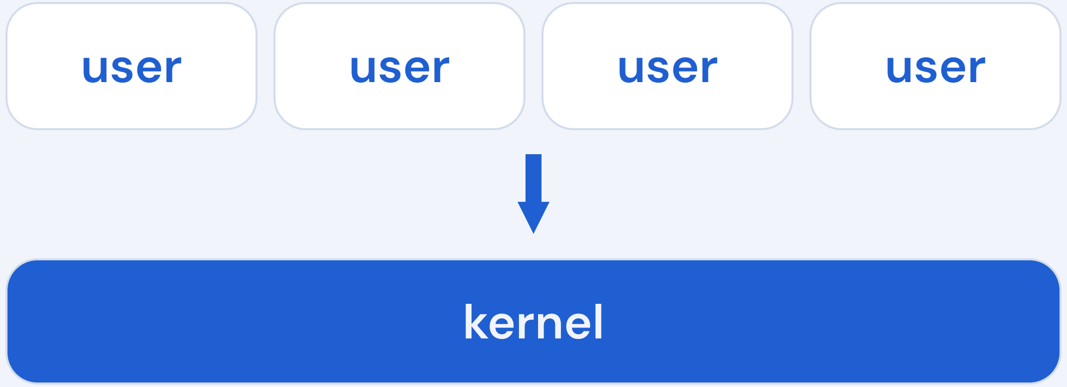
IN THE KERNEL

Kernel threads

Supported and managed directly by the operating system. Windows, Linux, macOS, iOS and Android all provide them.

A relationship must exist between the two. There are three common models.

Three models

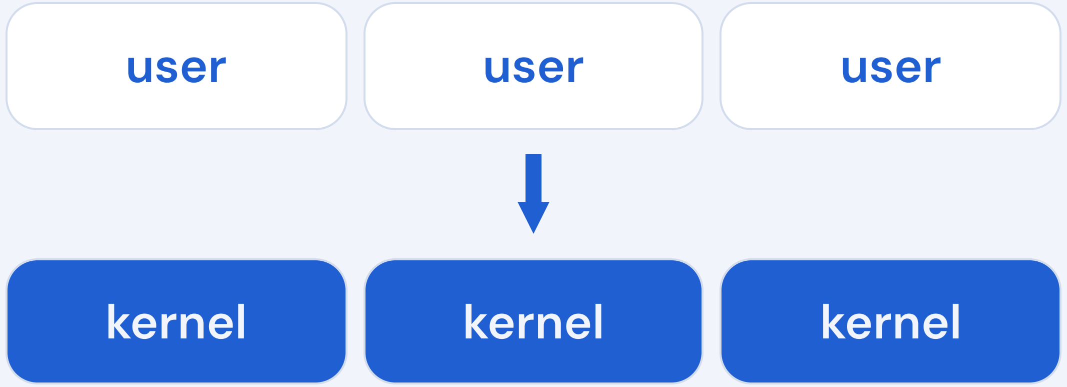


user user user user

kernel

Many-to-one

Many user threads on one kernel thread. If one blocks, all block, and they cannot run in parallel. Rarely used now.

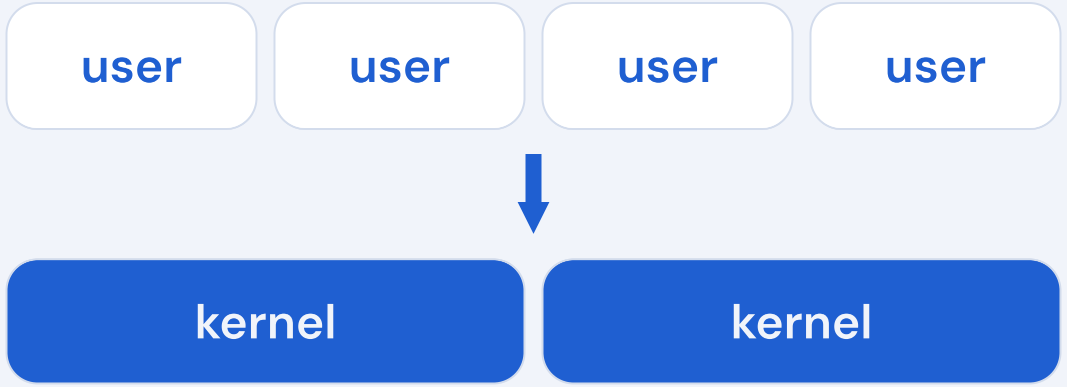


user user user

kernel kernel kernel

One-to-one

Each user thread has its own kernel thread. More concurrency, but each thread costs a kernel thread. Linux and Windows use this.



user user user user

kernel kernel

Many-to-many

Many user threads on a smaller or equal number of kernel threads. Flexible, but hard to implement.

The two-level model

What it is

Many-to-many, with one addition: a user thread may also be bound to its own kernel thread.

In practice

With many cores available, limiting the number of kernel threads matters less. Most operating systems now use one-to-one.

PART 4

Thread libraries

The API a programmer uses to create and manage threads.

Two ways to implement a library

NO KERNEL SUPPORT

Entirely in user space

All code and data of the library are in user space. Calling a library function is an ordinary function call, not a system call.

KERNEL SUPPORT

At kernel level

The library is supported directly by the operating system. Calling a library function usually results in a system call.

Pthreads may be either. Windows threads are kernel level. Java threads use whatever the host system provides.

Pthreads

```
#include <pthread.h>

void *work(void *arg) {
    int *n = arg;
    printf("thread got %d\n", *n);
    pthread_exit(0);
}

int main(void) {
    pthread_t tid;
    int value = 7;
    pthread_create(&tid, NULL, work, &value);
    pthread_join(tid, NULL);
}
```

A POSIX standard, IEEE 1003.1c.

It is a specification of behaviour, not an implementation. Common on UNIX, Linux and macOS.

pthread_create() starts a thread running the function work(). pthread_join() makes main() wait for it to finish.

Windows and Java threads

Windows

CreateThread() takes the function to run and a parameter for it, and returns a handle. WaitForSingleObject() waits for the thread, as pthread_join() does.

Java

Threads are managed by the JVM. A class implements the Runnable interface and its run() method, then a Thread object is created and start() is called. join() waits for it.

Since Java 5, the Executor framework separates creating threads from running tasks: we submit a task and receive a Future that will hold its result.

PART 5

Implicit threading

Let compilers and run-time libraries create and manage the threads.

Why implicit threading

The programmer identifies *tasks*, not threads.

The problem

As thread counts grow, getting a program correct with explicit threads becomes harder.

The idea

We say which pieces of work can run in parallel. The library decides how many threads to use and maps tasks to them.

Five approaches

Thread pools, fork-join, OpenMP, Grand Central Dispatch and Intel Threading Building Blocks.

Thread pools

Tasks arrive
requests to serve

→

Queue
tasks wait here

→

Pool of threads
created in advance

→

Done
thread returns to the pool

Faster

Serving a request with an existing thread is quicker than creating a new one.

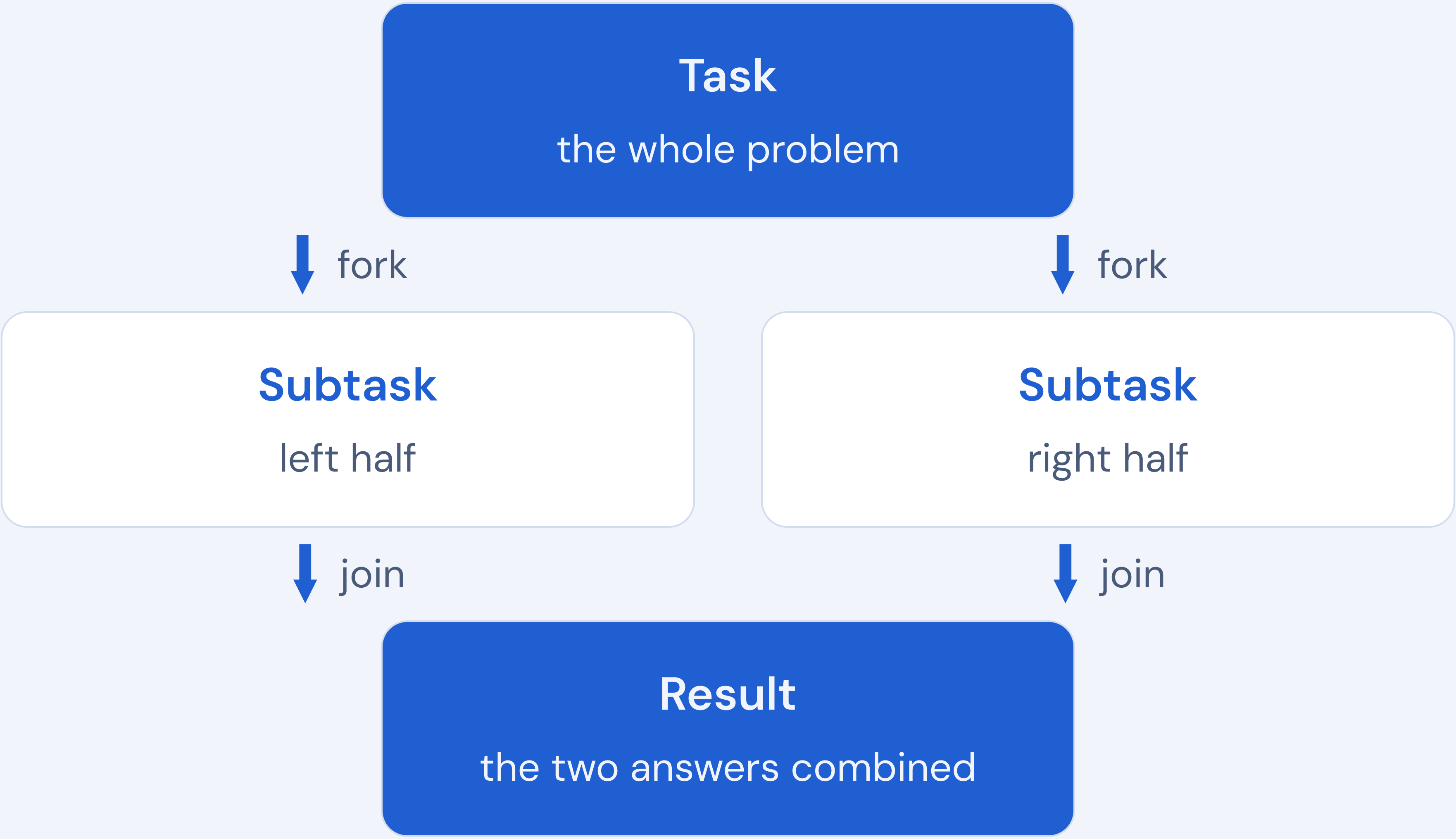
Bounded

The number of threads is limited to the size of the pool, so the system is not overloaded.

Flexible

The task is separate from how it is run, so it can be delayed or run periodically.

Fork-join parallelism



Task

the whole problem

fork

fork

Subtask

left half

Subtask

right half

join

join

Result

the two answers combined

If the problem is small enough, solve it directly. Otherwise split it, solve the parts as separate tasks, and combine. This suits divide-and-conquer algorithms.

OpenMP

```
#include <omp.h>

#pragma omp parallel for
for (int i = 0; i < n; i++)
    c[i] = a[i] + b[i];
```

Compiler directives and an API for C, C++ and Fortran.

The directive marks a parallel region: a block of code that may run in parallel.

Here the iterations of the loop are divided among as many threads as there are cores. We wrote one line.

Grand Central Dispatch and Intel TBB

Grand Central Dispatch

Apple's technology for macOS and iOS. We write blocks of work and place them on dispatch queues. A serial queue runs one block at a time, in order. A concurrent queue runs several at once. Queues have quality-of-service classes, from user-interactive to background.

Intel Threading Building Blocks

A template library for parallel programs in C++. We replace a loop by a parallel_for call and give it the range and the work for each element. The library divides the range among threads.

PART 6

Threading issues

Questions that have easy answers with one thread and harder ones with many.

fork() and exec() with threads

THE QUESTION

fork()

Does the new process copy all the threads, or only the one that called fork()? Some UNIX systems offer two versions of fork(), one for each answer.

NO CHANGE

exec()

It works as usual: the program named in the call replaces the whole process, including all of its threads.

If exec() is called right after fork(), copying all the threads is wasted work: copy only the calling thread.

Signal handling

A *signal* notifies a process that a particular event has occurred.

Three steps

An event generates the signal. The signal is delivered to a process. A handler deals with it: a default handler, or one the program defines.

One thread

The signal is delivered to the process. Simple.

Many threads

Deliver it to the thread the signal applies to, to every thread, to some threads, or to one thread appointed to receive all signals.

Thread cancellation

NOW

Asynchronous

The target thread is terminated immediately. It may be in the middle of updating shared data, or holding resources.

WHEN SAFE

Deferred

The target thread checks from time to time whether it should terminate, at cancellation points. It can clean up first. This is the default in Pthreads.

In Java, interrupt() sets a flag on the target thread, and the thread checks it with isInterrupted(). That is deferred cancellation.

Thread-local storage and scheduler activations

Thread-local storage

Each thread has its own copy of some data. Unlike a local variable, it is visible across function calls. Useful when we do not control thread creation, as with a thread pool.

Lightweight process

In the many-to-many and two-level models, an intermediate structure between user and kernel threads. To the thread library it looks like a virtual processor.

Scheduler activations

The kernel tells the application about certain events with an upcall, so the library can keep the right number of kernel threads.

Threads in Windows and Linux

THREADS

Windows

One-to-one. Each thread has an ID, a register set, a user stack and a kernel stack, and a private storage area. Three structures describe it: ETHREAD and KTHREAD in kernel space, and TEB in user space.

TASKS

Linux

Linux speaks of tasks, not threads. The clone() system call creates a task, and its flags decide how much the child shares with the parent: file system, memory, signal handlers, open files. Share everything and it behaves as a thread.

Seeing kernel threads: htop and the shell

```
htop
  H show the threads of each process
  K show the kernel threads
  F5 tree view: kthreadd is PID 2
  F4 filter by name

ps -L -o pid,lwp,nlwp,psr,comm -p PID
ps --ppid 2 -o pid,ppid,comm
```

In htop

H gives one row for each thread of a process. K shows the kernel threads. In the F5 tree, kthreadd (PID 2) is the parent of all of them.

From the shell

ps -L prints one row for each thread. LWP is the thread ID, NLWP the count, PSR the core it last ran on. The main thread's ID equals the PID.

Same facts, two views

htop is live and easy to browse. ps and /proc/PID/task/ give text you can save.

These tools only read. Never signal a kernel thread or press F9 on one. A container or WSL may hide them. Class Activity 3, Task 4 uses them.

Key takeaways

01

A thread is a unit of CPU use

Its own ID, program counter, registers and stack. Code, data and files are shared with the process.

02

Four benefits

Responsiveness, resource sharing, economy and scalability.

03

Amdahl's law

The serial part of a program limits what extra cores can give.

04

Think in tasks

Thread pools, fork-join and OpenMP manage the threads for us.

Next session

COMING UP

CPU scheduling

Which ready process or thread gets the CPU next, and how to compare scheduling algorithms.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 4.